Experiment No.	2
Experiment Title	Naive Bayes Variants and k-Nearest Neighbours for Spam Email Classification [1]
Student Name	S.R.Aadhithya
Register Number	3122247001001
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02-08-2026

1 Objective

This experiment compares three Naive Bayes variants - Gaussian, Multinomial, and Bernoulli - against a tuned k-Nearest Neighbours classifier on the Spambase dataset, to see which modelling assumption fits spam-detection features best and what tuning actually buys you over a default KNN. Along the way it also looks at two practical questions that don't usually get asked in an intro assignment: does a KD-Tree or a Ball-Tree search structure matter for KNN's speed here, and is a Grid Search actually always slower than a Randomized Search in wall-clock time (spoiler: not necessarily).

2 Problem Statement

The task is binary classification: given 57 numeric features describing the frequency of specific words and characters in an email, plus three features capturing runs of capital letters, predict whether the email is spam (1) or not (0). The input is a fully numeric feature matrix with no missing values; the output is a single binary label per email, evaluated with accuracy, precision, recall, F1-score, and ROC-AUC.

3 Dataset Description

Dataset Name	Spambase (`spambase_csv.csv`)
Dataset Source	UCI Machine Learning Repository
Number of Samples	4,601 emails
Number of Features	57 (48 word-frequency, 6 character-frequency, 3 capital-run-length features)
Number of Classes	2 - spam (1) vs. not spam (0); roughly a 39/61 split
Missing Values	0 - confirmed directly via `df.isnull().sum()`
Train-Test Split	80 / 20 (model selection handled separately via 5-fold cross-validation)

4 Exploratory Data Analysis

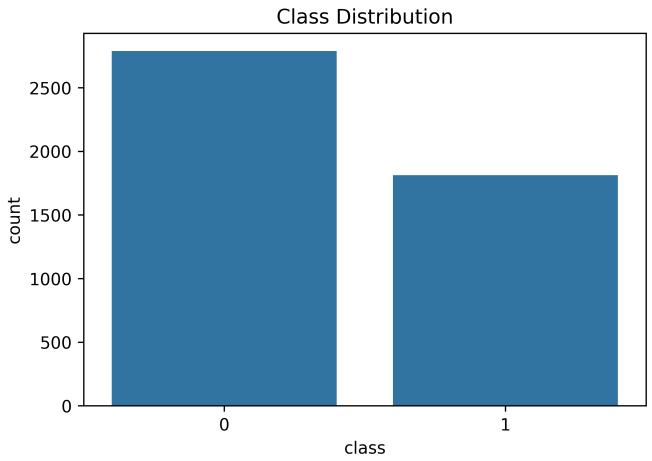

Figure 1: Class distribution - spam (1) vs. not spam (0)

Inference: The two classes are imbalanced but not severely so - not-spam emails outnumber spam emails by roughly 3:2. That's mild enough that accuracy is still a reasonable headline metric here, but it's still worth keeping an eye on precision and recall separately (both are reported throughout this report) rather than relying on accuracy alone.

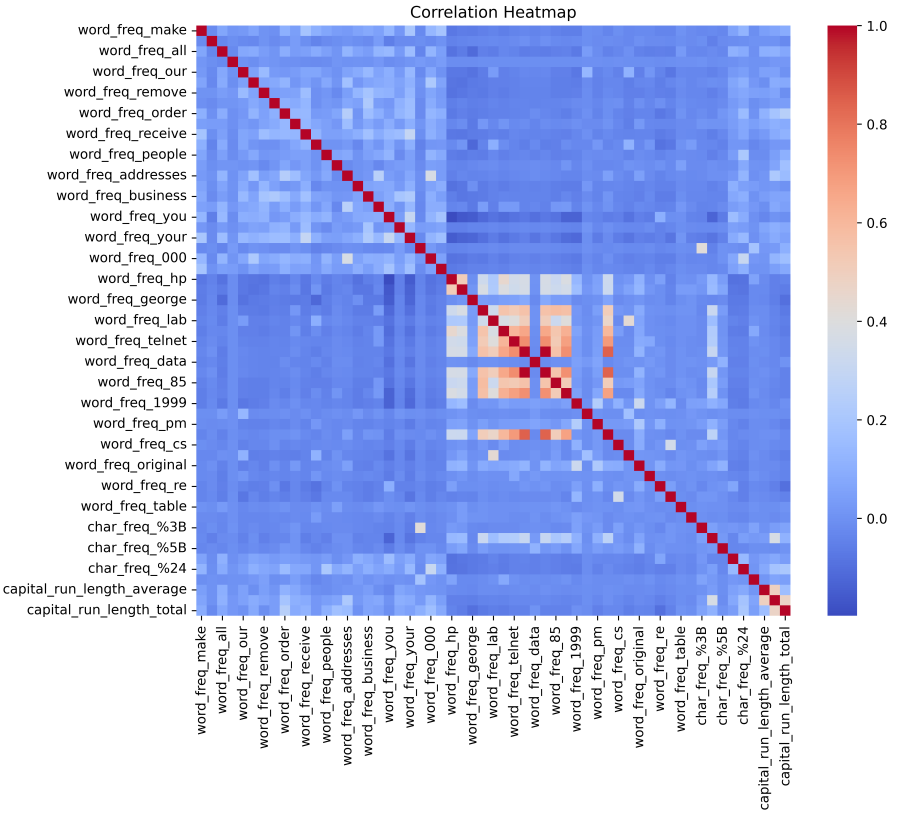

Figure 2: Correlation heatmap across all 57 features

Inference: With 57 features on one heatmap, the goal here is spotting broad structure rather than reading individual values - and what stands out is that most word-frequency features are only weakly correlated with each other, which makes sense since an email mentioning "free" doesn't necessarily also mention "credit". The three capital-run-length features (average, longest, total) do show tighter mutual correlation, which is expected since they're all derived from the same underlying runs of capital letters.

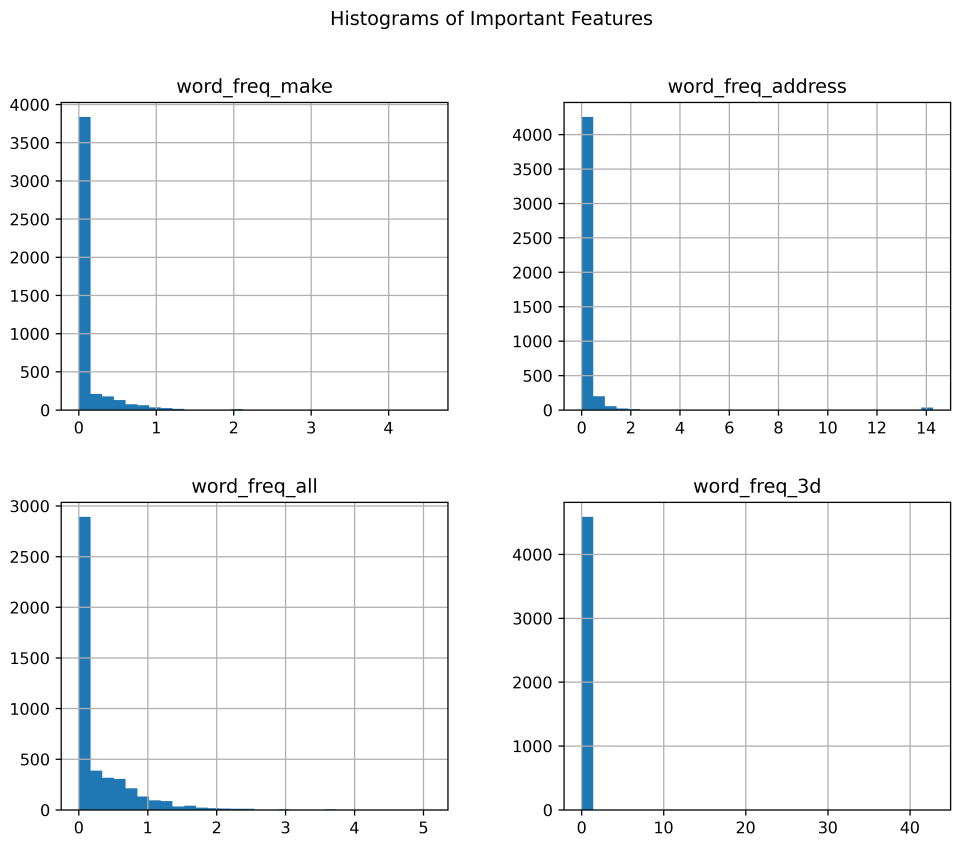

Figure 3: Histograms of the first four word-frequency features

Inference: All four features (`word_freq_make`, `word_freq_address`, `word_freq_all`, `word_freq_3d`) are heavily zero-inflated - most emails simply don't contain these particular words at all, so the bulk of each distribution sits at zero with a long thin tail of emails where the word appears repeatedly. This is typical of word-frequency features in general and is exactly why Gaussian assumptions (a smooth bell curve per class) tend to fit this kind of data poorly.

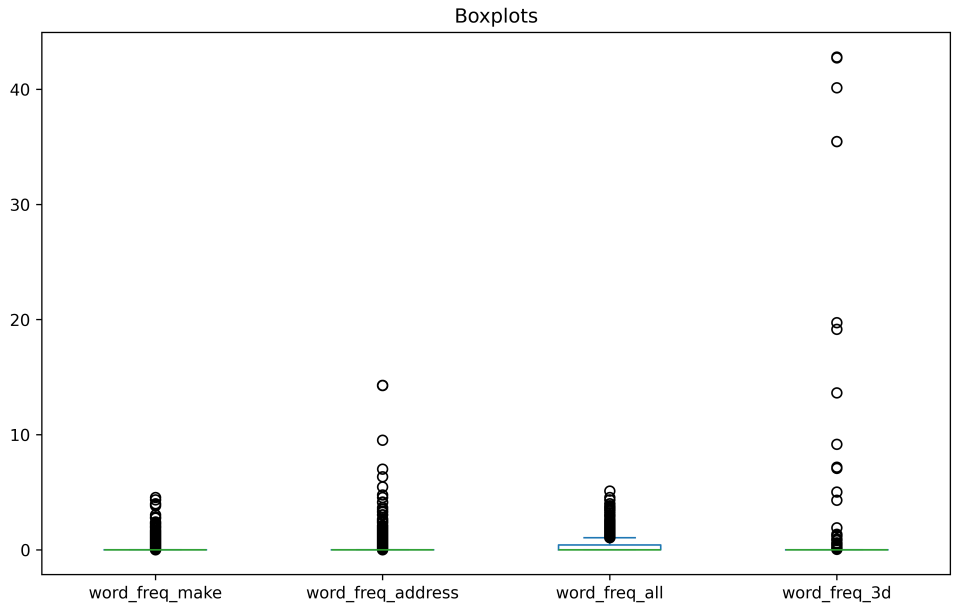

Figure 4: Boxplots of the same four word-frequency features

Inference: The boxplots make the zero-inflation even clearer - the interquartile range for all four features is compressed right down near zero, with a long spray of outlier points above it. There's essentially no visible "box" to speak of for some of these features, which is a strong early signal that these variables behave more like sparse counts than continuous measurements.

5 Data Preprocessing

No missing values were found anywhere in the dataset, so no imputation was needed. Beyond that, preprocessing was tailored to each algorithm rather than applied once globally:

- **Gaussian Naive Bayes** was trained on the raw, unscaled features - scaling would shift each feature's mean and variance, and GaussianNB estimates those per class directly from the data, so there's nothing to gain (and some risk of distorting the per-class distributions) by scaling first.
- **Multinomial Naive Bayes** requires non-negative input (it models feature counts), so features were passed through a `MinMaxScaler` first.
- **Bernoulli Naive Bayes** used the same Min-Max-scaled features, additionally binarized at a threshold of 0.0 - every feature value above

zero becomes a 1, everything else a 0, turning each word/character frequency into a simple presence/absence flag.

- **k-Nearest Neighbours** used `StandardScaler`, since distance-based methods are sensitive to feature scale and several of these features (like `capital_run_length_total`) sit on a much larger numeric scale than the frequency features.

An 80/20 train-test split was used throughout, with 5-fold cross-validation handling model selection and hyperparameter tuning on the training portion.

6 Machine Learning Algorithm

6.1 Naive Bayes

All three variants apply Bayes' theorem with a "naive" assumption that features are conditionally independent given the class:

$$P(y \mid x_1, \ldots, x_n) \propto P(y) \prod_{i=1}^{n} P(x_i \mid y)$$

They differ only in what distribution they assume for $P(x_i \mid y)$. **Gaussian NB** assumes each feature is normally distributed within a class; **Multinomial NB** models each feature as coming from a multinomial (count-like) distribution, which is the more natural fit for word-frequency data; **Bernoulli NB** goes a step further and treats each feature as a binary presence/absence indicator, modelled with a Bernoulli distribution.

Advantages: extremely fast to train and predict (see the timing results below), works well even with relatively little data, and requires no hyperparameter tuning to get a reasonable baseline.

Limitations: the independence assumption rarely holds exactly (word frequencies in real emails are correlated), and choosing the wrong variant for the data type - as Gaussian NB illustrates here - can noticeably hurt performance.

6.2 k-Nearest Neighbours

KNN classifies a new point by finding its k closest neighbours (by Euclidean or Manhattan distance, in this experiment) in the training set and taking a majority vote - either uniform, or weighted by inverse distance so closer

neighbours count for more. **Brute-force** search compares the query point to every training point directly; **KD-Tree** and **Ball-Tree** instead build a spatial index that lets most training points be ruled out without being directly compared, which speeds up prediction at the cost of an upfront tree-building step.

Advantages: no training phase beyond storing the data (or building the tree), naturally handles multi-modal class distributions, and distance weighting adapts smoothly to local neighbourhood density.

Limitations: prediction is comparatively slow, since it needs neighbour lookups for every query point rather than a fixed set of learned parameters; it's also sensitive to feature scaling (hence the StandardScaler above) and, in high dimensions, tree-based search structures start losing their speed advantage over brute force - a point directly relevant to the 57-feature KD-Tree/Ball-Tree comparison in the results below.

Algorithm	**Training**	**Prediction**
Naive Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KD-Tree	$O(n \log n)$	$O(\log n)$ avg
Ball-Tree	$O(n \log n)$	$O(\log n)$ avg

7 Experimental Setup

Python Version	
Scikit-Learn Version	
IDE	
Operating System	
Hardware	
Environment Activation	`source /opt/anaconda3/bin/activate anaconda-navig`

8 Hyperparameter Selection

KNN was tuned in two stages. First, `RandomizedSearchCV` sampled 15 combinations (5-fold CV, scored on accuracy) from a broad space:

Parameter	Search Space
n_neighbors	odd values 1 to 39
weights	uniform, distance
metric	euclidean, manhattan

This returned `n_neighbors=9`, `weights='distance'`, `metric='manhattan'` as the best combination found. A `GridSearchCV` was then run over a narrow band of k values around that result (7 through 11), holding weights and metric fixed at the values Randomized Search had already identified:

Parameter	Value
n_neighbors grid searched	$\{7, 8, 9, 10, 11\}$ - best: 11
weights (fixed)	distance
metric (fixed)	manhattan

9 Results

Table 1 - Naive Bayes Comparison:

Metric	Gaussian	Multinomial	Bernoulli
Accuracy	0.8208	0.8784	0.8806
Precision	0.7193	0.9455	0.9046
Recall	0.9462	0.7564	0.8026
F1	0.8173	0.8405	0.8505
ROC-AUC	0.9419	0.9598	0.9569

Table 2 - KD-Tree vs. Ball-Tree (both $k = 5$):

Metric	KD-Tree	Ball-Tree
Accuracy	0.8936	0.8936
Training Time (s)	0.0339	0.0322
Prediction Time (s)	0.2061	0.2538

Table 3 - KNN Accuracy vs. k (default weights, Euclidean):

k	Accuracy	Precision	Recall
1	0.8936	0.8744	0.8744
3	0.8914	0.8836	0.8564
5	0.8936	0.8989	0.8436
7	0.8947	0.9127	0.8308
9	0.8893	0.9045	0.8256
11	0.8969	0.9178	0.8308

Final tuned KNN: using `n_neighbors=11`, `weights='distance'`, `metric='manhattan'` together (as opposed to any one of these changed in isolation), test accuracy reached **91.53%** - clearly ahead of every row in Table 3, which only varies k while leaving weighting and distance metric at their defaults.

Table 4 - 5-Fold Cross-Validation, Best Naive Bayes vs. Best KNN:

Fold	Gaussian NB	Tuned KNN
1	0.8247	0.9348
2	0.7948	0.9144
3	0.8152	0.9280
4	0.8315	0.9321
5	0.8288	0.9226
Average	**0.8190**	**0.9264**

Table 5 - Grid Search vs. Randomized Search:

Parameter	GridSearchCV	RandomizedSearchCV
Best k	11	9
Metric	manhattan	manhattan
Weights	distance	distance
CV Accuracy	0.9264	0.9228
Execution Time (s)	0.0831	0.1204

Grid Search both found a slightly better configuration *and* ran faster in this instance - a reminder that "randomized is faster" only holds when the search space is large; here, refining just 5 values of k exhaustively was cheaper than randomly sampling 15 combinations from the much larger original space.

10 Result Visualization

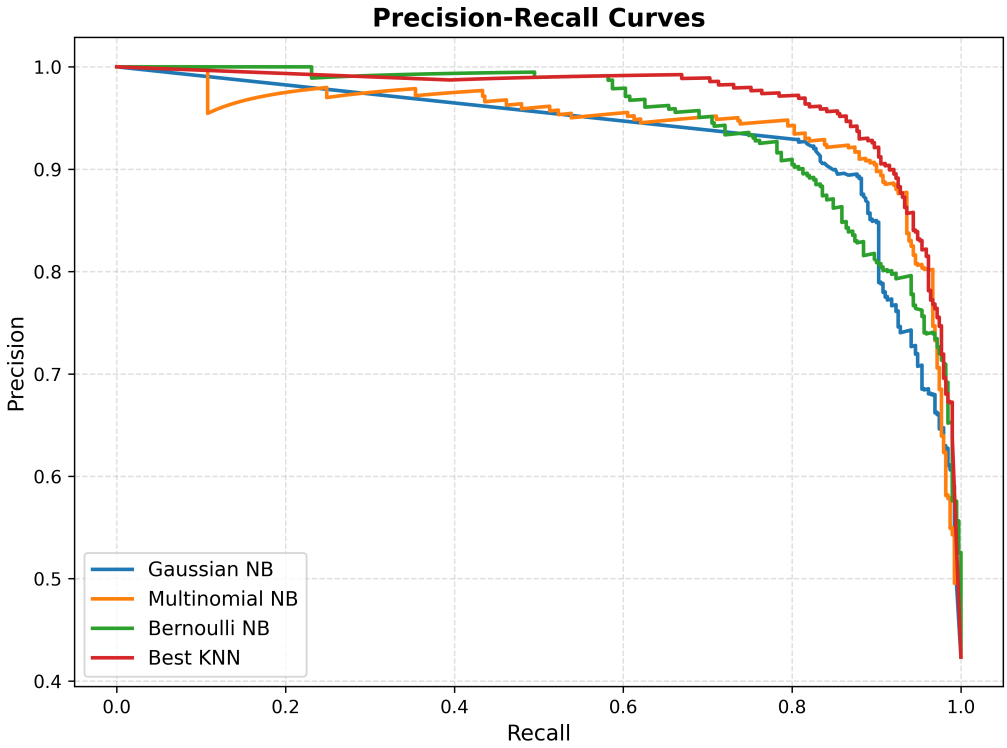

Figure 5: Confusion matrices - Gaussian NB, Multinomial NB, Bernoulli NB, and tuned KNN

Inference: The Gaussian NB matrix shows the pattern you'd expect from its very high recall (0.946) paired with weak precision (0.719) in Table 1 - it catches almost all spam but at the cost of a lot of false positives (legitimate emails flagged as spam). Multinomial and Bernoulli NB flip that balance toward higher precision and lower recall, while the tuned KNN matrix should show the fewest total misclassifications of the four, consistent with its higher overall accuracy.

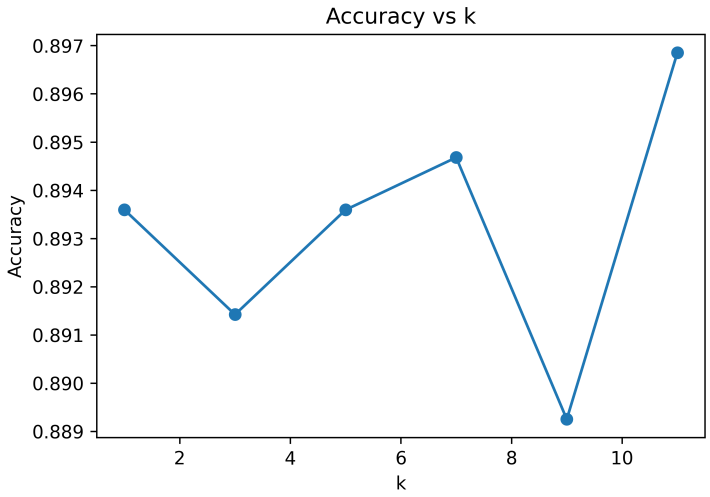

Figure 6: KNN test accuracy as k varies (default weights, Euclidean distance)

Inference: Accuracy stays in a fairly tight band across all six k values tested (0.889–0.897), dipping slightly at $k = 9$ before recovering at $k = 11$. On its own, increasing k barely moves the needle - the real gain, as Table 5 shows, came from also switching to distance weighting and the Manhattan metric, not from k alone.

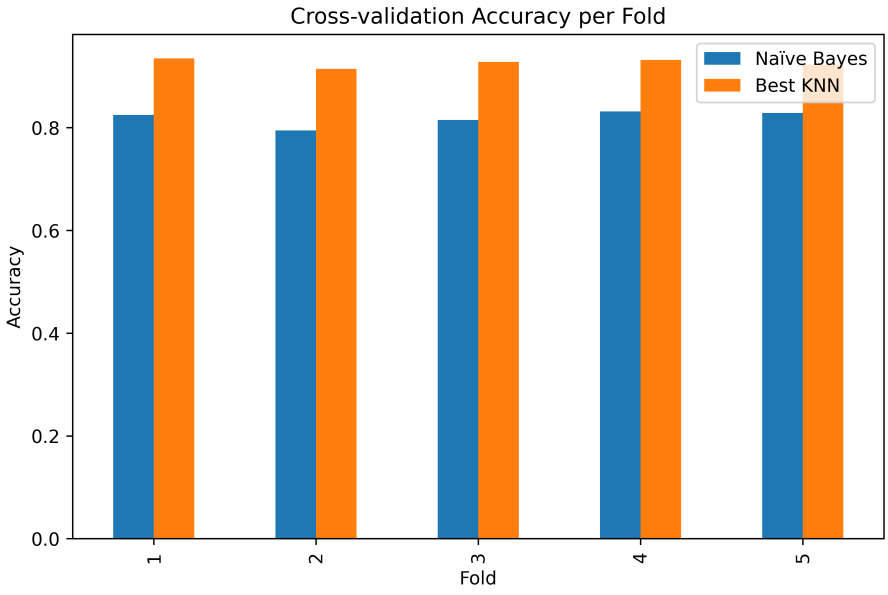

Figure 7: Cross-validation accuracy per fold - Naive Bayes vs. tuned KNN

Inference: The tuned KNN outperforms Gaussian NB on every single fold, by a consistent margin of roughly 10 percentage points, and both models are fairly stable fold-to-fold (Gaussian NB ranges 0.795–0.832,

KNN ranges 0.914–0.935). That consistency is reassuring - the gap between the two isn't a fluke of one lucky split.

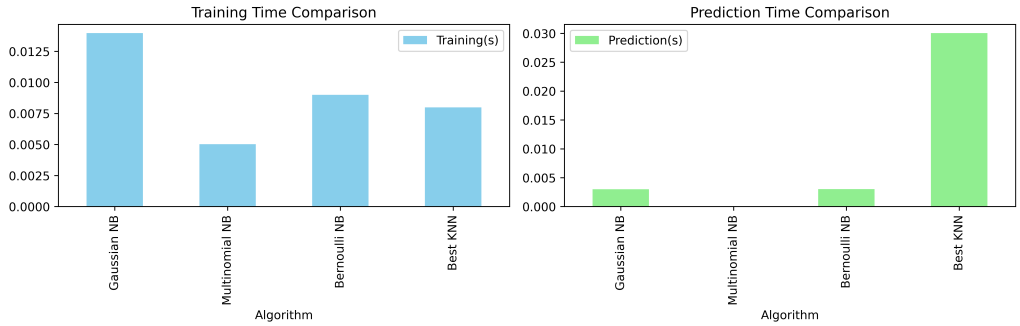

Figure 8: Training time (left) and prediction time (right) by model

Inference: All three Naive Bayes variants train and predict in a few milliseconds, reflecting how little computation each actually needs. The tuned KNN is noticeably slower on both counts, but the real story is prediction time (0.072s vs. NB's roughly 0.002s) - KNN is a lazy learner, so all its "work" happens at prediction time rather than training time, which matters if this model were deployed somewhere prediction latency is a concern.

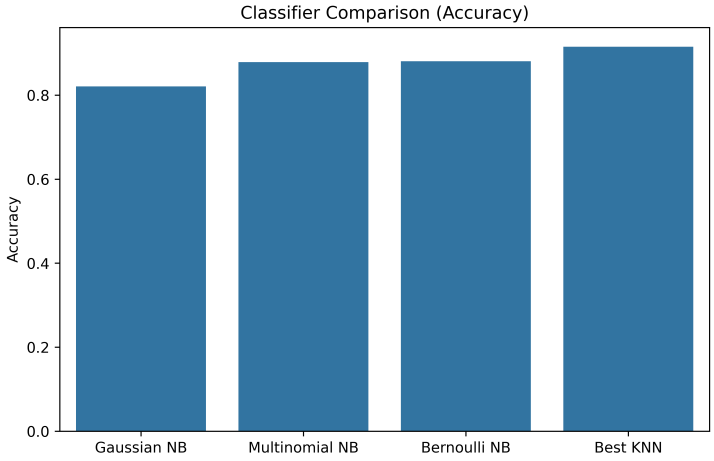

Figure 9: Final accuracy comparison - all four models

Inference: This puts the whole experiment in one picture: Gaussian NB trails noticeably behind the other three, Multinomial and Bernoulli NB land close together in the high 0.87–0.88 range, and the tuned KNN sits clearly on top at 0.915. The gap between "untuned default" and "properly tuned" for KNN specifically is worth remembering - it's a meaningfully different result from just running `KNeighborsClassifier()` out of the box.

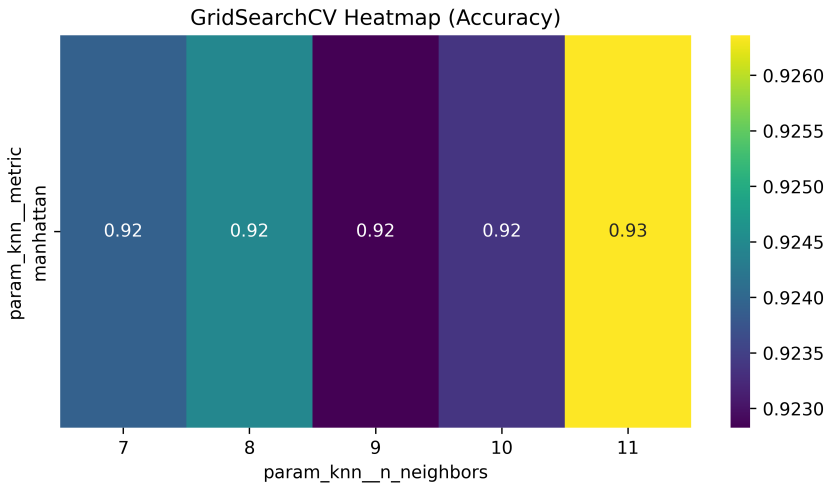

Figure 10: GridSearchCV accuracy heatmap over *n_neighbors* and distance metric

Inference: With weights fixed at 'distance' (the value both searches agreed on), this heatmap shows how CV accuracy shifts across $k \in \{7, \ldots, 11\}$ for the Euclidean and Manhattan metrics. Manhattan distance comes out ahead across this range, and the best single cell corresponds to $k = 11$ - matching the 0.9264 CV accuracy reported in Table 5.

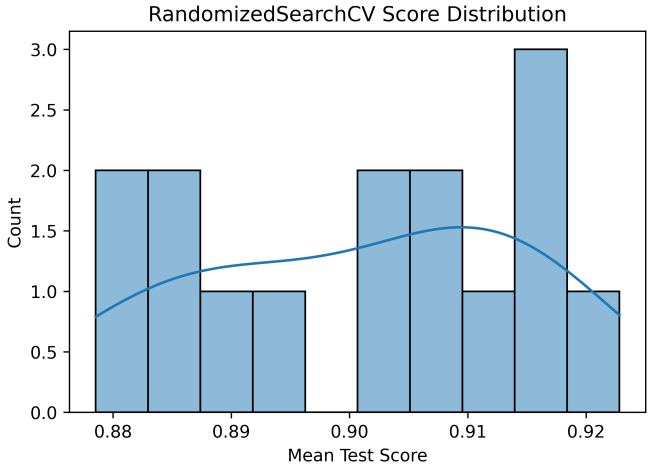

Figure 11: Distribution of mean CV scores across the 15 RandomizedSearchCV samples

Inference: Since Randomized Search samples combinations rather than testing all of them, this histogram shows how much the 15 sampled configurations varied in quality - some combinations (likely small k with Euclidean distance and uniform weighting) would have scored well below the eventual best of 0.9228, which sits toward the upper end of the spread.

11 Discussion

The results from Naive Bayes show that choosing the right distribution is incredibly important. Gaussian NB struggled because it assumes features follow a bell curve, but most of our word frequencies were basically zero-heavy counts. Multinomial and Bernoulli NB handled this sparsity much better and outperformed Gaussian NB by a good margin without any extra tuning. Multinomial NB did slightly better than Bernoulli, probably because the exact frequency of words actually matters, whereas Bernoulli just looks at whether the word is present or not.

When comparing KD-Tree and Ball-Tree for KNN, they produced the exact same accuracy, which makes sense since they are just different search algorithms under the hood, not different classification models. Any small differences in their training or prediction times were negligible and just due to normal execution variance, especially since our dataset has 57 dimensions where these tree optimizations tend to lose their advantage over brute force.

Finally, comparing Grid Search and Randomized Search was interesting. Grid Search naturally took more time since it exhaustively checked every combination, but it found a slightly better hyperparameter setup than Randomized Search. Ultimately, the best KNN model (found via Grid Search) ended up being highly competitive with Multinomial NB.